WERGONIC · QUALITY

QA Learning Roadmap

What to learn, in what order, and why it matters — written for someone starting from zero, in a world where an AI can write the test code but cannot decide what to test.

For — our new QA

Prepared by — Amine Khiati

Date — 3 September 2026

Read alongside — QA Onboarding & Study Guide · QA Testing Checklist · Product & Technical Documentation

How to read this

A map, not a syllabus. Roughly six months of part-time study, alongside real work from day one.

This document is a list of things worth understanding, in the order that makes each one easier than the last. Every section says three things: what the topic actually is, why it matters for our product specifically, and how you'll know you've got it. Where a resource is genuinely worth your time, it's named in the text. Where it isn't, it isn't mentioned — that's deliberate.

Two ideas run through the whole thing, and both are worth stating before you start.

The first: you will be useful in week one, long before you finish this roadmap. Using our products carefully and reporting what's wrong is real, valuable work that needs no technical knowledge at all. Everything here makes you better at it — none of it is a gate you have to pass before you're allowed to contribute.

The second: AI has genuinely destroyed parts of this profession, and pretending otherwise would waste your time. It has not touched the other parts. This whole roadmap is organised around that line. Everything in it is something AI cannot do for you, and there's a section near the end listing what to deliberately not study, with the reason for each.

> **Order matters.**
>
> The classic mistake is jumping straight to writing automated tests, because that's the part that looks like the job. Test code you don't understand is worse than no test code — it fails randomly, everyone learns to ignore it, and the whole suite dies. Understand the system first.

PART ONE

The job before the tools

Months one and two. None of this requires you to write a line of code.

1.1 What quality work actually is

The word "testing" makes the job sound like clicking every button and confirming it works. That's a small part of it. The real work is **deciding what could go wrong and going to look**.

Think of it as three roles. Product decides what we should build. Developers decide how it gets built. You decide whether we actually believe it works — and you're the only person in the chain whose job is to be suspicious. Developers test that their feature works; that's a different question from whether it survives a confused user, bad data, a dropped connection or the wrong account.

This distinction is the whole reason the role still exists after AI. Writing test code used to be the slow, expensive part, so that's what the industry hired for. It isn't slow or expensive any more. Deciding what deserves a test, what counts as broken, and whether a passing test proves anything — that is untouched, and it's the skill you're building.

1.2 How our system fits together, end to end

This is the highest-value week you will spend. Almost every question you'll be asked reduces to "which part of the system is lying?", and you can't answer that without a mental picture of the parts.

Ours, simplified: sensors are attached to a worker and talk over Bluetooth to **the mobile app**, which records a session. The app sends that data to **the backend** — a server that stores it in a database and runs the calculations that turn raw movement into ergonomic results. **The web panel** is what our customers open in a browser to see those results. Four moving parts, and a bug can live in any one of them.

What to actually learn, in plain terms: the difference between the client (the app or page in front of you) and the server (the machine that holds the truth); what an API is, and why the two talk in requests and responses; what JSON looks like; and what the common status codes mean, because they tell you who to blame. Roughly: 200 means fine, 400-ish means the app sent something wrong, 401 and 403 mean you're not logged in or not allowed, 404 means it doesn't exist, and anything 500-ish means the server itself broke — that one is always worth a ticket. Then: what a database table and a row are, and the difference between *authentication* (who are you) and *authorisation* (what are you allowed to see). MDN's "How the web works" covers the ground in an afternoon; the rest comes from using our own product with the browser's network tab open next to it.

> **YOU'VE GOT IT WHEN**
> you can look at a broken screen and say "the server returned an error" or "the server was fine, the page just didn't draw it" — without asking a developer.

1.3 Reading evidence instead of describing symptoms

Any QA can say "the report page is broken". What earns you respect is: "opening a report for a session with no calibration returns a 500; Sentry shows a missing trunk angle; it works for every session that has calibration."

Same bug, but the second version gets fixed the same afternoon and the first one costs a developer an hour of guessing. The tools that get you from one to the other are ordinary and free: the browser's developer tools — the network tab shows every request the page made, what it sent and what came back, and the console shows the errors the page hit — plus **Sentry**, where our applications automatically report their crashes, and the server logs when you need to go deeper.

Get into the habit of opening developer tools *before* you start testing, not after something breaks. A good share of the bugs you'll find are visible there minutes before they're visible on screen.

> **YOU'VE GOT IT WHEN**
> every bug you file names the failing request and carries the actual error text, not a description of what you saw.

1.4 Reproducing and narrowing

A bug you can't reproduce is a rumour. A bug you can reproduce but haven't narrowed is a chore you've handed to someone else.

The method is simple and mechanical. Reproduce it a second time to prove it's real. Then change one variable at a time and see whether it still happens: a different account, a different organisation, a different role, a different browser, a different phone, an older session, staging instead of production. Each variable you eliminate makes the report sharper. What you're hunting for is the **boundary** — the sentence that starts "it happens when… and doesn't happen when…". That sentence is usually most of the diagnosis.

If after a real attempt it only happened once, say exactly that. An honest "seen once, couldn't reproduce, here's the screenshot" is useful. A confident report that wastes someone's day is not.

1.5 Test design — the skill AI hasn't touched

This is the centre of the roadmap. Test design is choosing what to try, and it survives every wave of tooling because it's about our product's intent rather than about code.

There's a formal vocabulary for it, and two terms are worth knowing properly. **Equivalence partitioning** is the realisation that most inputs fall into groups where every member behaves the same, so instead of trying a hundred values you try one from each group. **Boundary value analysis** is the observation that bugs cluster at the edges of those groups — zero, one, the maximum, one past the maximum, empty, the first and last day of a month, the exact moment a session ends. Between them, these two ideas find a large share of all real defects, and you can learn both in an evening.

Beyond that, the useful thinking is a set of habits. Ask what happens if the steps are done in a different order, or interrupted halfway — network lost, app backgrounded, page refreshed, submit pressed twice. Ask what happens with the wrong role or the wrong organisation, on every screen. Ask what the feature does with no data at all, and with far more data than anyone expected. Ask what a careless or confused person would do, because they're your real customers, not the careful ones. Elisabeth Hendrickson's one-page "Test Heuristics Cheat Sheet" collects most of this better than any course — print it and keep it beside you for the first months.

> **YOU'VE GOT IT WHEN**
> you can be handed a new feature and produce, in ten minutes, a short list of the things most likely to break it — and a decent share of them turn out to be real.

1.6 Where code lives and how it reaches you

You need enough of this to know what you're testing and what "fixed" means, and no more.

Our code lives in repositories. Work happens on branches and gets merged after review. Two branches matter: `dev`, which automatically deploys to **staging** — our private copy of the product — and `main`, which deploys to **production**, the live system our customers use. Almost all of your testing happens on staging, and you never run experiments against production data.

The practical consequence is that a bug report without an environment is incomplete, and "the developer fixed it" does not mean the customer has it. A fix on `dev` reaches staging in minutes and production only at the next release — which is why re-checking the main flows after each release is part of your job, not an optional extra. "Learn Git Branching" gets you the mental model in an afternoon; you won't need much more than that for a long while.

PART TWO

Enough technical depth to be dangerous

Months two to four. The goal is not to become a developer — it's to stop needing one for every question.

2.1 Reading code, not writing it

There's a version of "learning to code" that takes years, and a version that takes a couple of months of evenings. You want the second: the ability to open a file, follow what a function does, and form an opinion about it.

Three reasons it pays off, all of them immediate. It lets you tell a display problem from a data problem by looking. It lets you find where a rule is actually implemented and check it against what product said it should be — a surprising number of arguments about "is this a bug" get settled that way. And, critically, it lets you review the test code AI writes for you instead of trusting it.

Our web side is TypeScript and React; our backend is Python and Django. Start with JavaScript and TypeScript basics, since that's where you'll be reading most often, then Python a month or two later. freeCodeCamp is fine, free and enough. Read far more code than you write at this stage, and read *our* code — it's about our product, which doubles the value of every hour.

2.2 Data — SQL, at the level of asking questions

The screen shows you what the app *thinks*. Sooner or later you need to know what was actually stored, and for that you go to the database directly.

You need a genuinely small slice of SQL: `SELECT`, `WHERE`, `ORDER BY`, `COUNT`, `LIMIT`, and eventually `JOIN` to follow a session back to its worker and organisation. A weekend is enough to be functional, and you'll never need the rest.

> **Read-only, always.**
>
> Our access to production data is read-only and stays that way. You are there to look, never to change. If a test needs data created or altered, that happens on staging.

What this buys you is the ability to write the strongest sentence in bug reporting: "the value was stored correctly; the API is returning it wrong." That sentence ends debates.

2.3 Talking to the API directly

Everything the web panel and the mobile app can do, they do by calling the backend. You can make those same calls yourself, with Postman or `curl`, with no interface in the way.

Two things become possible once you can. First, you separate frontend bugs from backend bugs in seconds instead of arguing about them. Second — and this is the big one — you can test the rules the interface hides from you. The interface only offers the buttons you're allowed to press, so clicking around can never prove that the rules behind those buttons are actually enforced.

2.4 Permissions and data boundaries

Take this section more seriously than its length suggests.

We are a multi-organisation product: many customer companies live in the same system, and each must see only its own workers, sessions and results. That guarantee is enforced by code on the backend, and code can be wrong. We have already had a real case where data crossed an organisation boundary and was visible to people who should never have seen it.

You cannot find that class of bug by clicking around as one user, because the interface will never offer you the wrong data. You find it by taking one account's access token, asking the API directly for another organisation's data, and checking that you're refused. Get our role model from the technical documentation, then work through it methodically: for each role, what should it be able to read, change and delete — and what does the API actually allow. This is slow, unglamorous, and among the most valuable work you will do here.

2.5 Automated tests — what they are, and which are worth having

An automated test is a small piece of code that exercises part of the product and checks the result, so a machine can repeat in seconds what you'd otherwise repeat by hand every week. They come in three sizes, and the difference between them is the whole game.

A **unit test** checks one function's logic in isolation — give it these numbers, expect that answer. Milliseconds to run, trivial to debug when it fails, because it can only be one thing. Our ergonomic calculations are the obvious home for these: precise rules, precise expected answers, and expensive to get wrong.

An **API test** calls a real endpoint and checks both the response and what ended up in the database. This is the best value for money in our situation: it catches broken validation, permission holes, and changes that silently break the mobile app, while still running fast and failing in a way that points somewhere specific.

An **end-to-end test** drives a real browser like a user — log in, click through, check what's on screen. It's the most convincing kind and the most expensive to keep alive: slow, and it breaks whenever the interface moves. You want a handful of these, covering only the flows that must never break.

Hence the rule you'll hear called "the testing pyramid": many small tests, some middle ones, very few big ones. The reason isn't tradition. It's that a failing small test tells you exactly what's wrong, and a failing end-to-end test only tells you that something, somewhere, is.

2.6 What makes a test worth keeping

This is the section beginners skip, and the one that decides whether your suite becomes an asset or a liability. **A test that fails randomly is worse than no test at all**, because within a month everyone has learned to ignore red — including on the day it's telling the truth.

Four properties to understand, and then to check for in everything you or an AI writes. **Determinism**: the same test on the same code gives the same answer every time. If it depends on today's date, on what happens to be in the database, on a random value, or on how fast the machine is, it will eventually lie to you. **Isolation**: a test creates the data it needs and doesn't care what ran before it — tests that must run in a particular order rot fast; on the backend `factory-boy` is how we create that data cleanly. **Waiting properly**: in browser tests, never pause for a fixed number of seconds, wait for the specific thing you expect to appear; fixed sleeps are the single largest source of flaky tests in the industry. **Real assertions**: a test that runs some code but checks nothing meaningful will pass forever, including when the feature is completely dead.

> **The habit that catches all four.**

> Before you trust a new test, break the feature on purpose and confirm the test goes red. A test you have never seen fail is not a test yet — it's a hope. This takes thirty seconds and it is the single most important habit in this document.

2.7 Our actual tools

Learn the tool that matches what you're testing that month. Don't try to learn them all upfront.

On the **backend** it's Python with Django's test framework; `pytest` and `factory-boy` are available and are where the value is for calculation and API tests. On the **web** it's `vitest` with Testing Library for component-level checks; full-journey browser tests would use **Playwright**, which we don't have yet and which is a natural project for you once you're comfortable. On **mobile** it's Flutter's own test framework — and the mobile app is by far our best-tested product, with several hundred maintained tests. Read them early. They're a working example of everything in section 2.6, written against our own domain.

2.8 The pipeline

Every time a developer pushes code, a robot wakes up on GitHub Actions and does the same sequence: fetch the code, install everything, check the types, run the linter, build the applications, run the tests — and only if all of that passes, package the result and ship it to our servers. That's CI/CD, and "the build is red" means one of those steps failed.

What you need from it is, first, the ability to open a failed run and read far enough to say whether it's a genuine failure, a flaky test, or infrastructure having a bad day. Second, over time, the ability to add your own tests to it — because a test that isn't in the pipeline only protects us on the days someone remembers to run it.

> **Where we honestly stand.**
>
> The mobile app is well covered. The backend has very little real coverage, and the web applications have close to none — the pipeline runs the tests, but there's almost nothing there to run. So today a green build proves the code compiles and passes the linter, and not much beyond that. Changing that is the medium-term point of your job, and it's why we're extending the pipeline right now: to make green mean something.

PART THREE

The parts that are ours alone

Ongoing, from month one. No course covers any of this — which is exactly why it's where you become hard to replace.

3.1 The domain: bodies, angles and exposure

We measure how people move at work and turn that into ergonomic assessments. That means our product is full of rules — how an angle is measured, what counts as a risky posture, how a session is calibrated, how exposure is scored over time — and those rules come from ergonomics research, not from intuition.

The practical consequence is that **a lot of what looks wrong is intentional**. A band that seems to start at the wrong number, a threshold that looks generous, a value that changes after calibration: several of these are deliberate decisions with a reason behind them. Filing them as bugs costs the team time and costs you credibility. Read the calculations document and the product documentation before you report anything in this area, and when something looks odd, ask first — "is this intended?" is a perfectly good question and nobody will think less of you for asking it.

Once you know the rules well, the reverse becomes true: you'll start catching real calculation errors nobody else is positioned to see, because you'll be the only person systematically comparing what the code does against what the rules say. That's a genuinely senior contribution, and it's reachable within your first year.

3.2 Hardware, mobile, and the real world

Sensors are attached to a moving human, connected over Bluetooth, powered by a battery, feeding a phone that may lock, sleep, fill up its storage or lose its network — in a workplace, not at a desk. This is the part of our product where automation helps least and a careful person is irreplaceable.

Test it the way it will really be used, and be deliberately hostile. Walk out of range mid-session and come back. Turn a sensor off. Let a battery run low. Put the phone in flight mode while recording and take it out again ten minutes later. Background the app, take a call, come back. Kill the app outright and reopen it. Then check that what ended up stored matches what actually happened.

Keep notes of timings — how long a reconnection takes, how long before a session is treated as lost. Those numbers are product decisions, and half the time nobody has written them down anywhere except in a QA's notebook.

3.3 The non-functional basics, in their light version

Three areas where you want awareness rather than expertise, at least for the first year.

Performance: notice when something is slow, and be specific about which thing and with how much data. A page that's fine with ten sessions and unusable with a thousand is a real bug, and usually only you will find it, because developers test with small data. **Accessibility:** can the interface be used with a keyboard alone, is the contrast readable, are form fields labelled — the obvious failures take minutes to catch. **Security thinking:** not

penetration testing, just the reflex of changing an ID in a URL to someone else's and seeing what happens. That reflex, applied consistently, finds more real problems than most tools do.

PART FOUR

Working with AI without ending up useless

You'll use Claude every day here. The difference between it making you fast and it making you replaceable is entirely in how you use it.

Start from an honest view of what it's good at. It's excellent at turning a scenario *you* designed into working test code, at explaining unfamiliar code line by line, at drafting boring scaffolding, at reading a wall of logs and pointing at the interesting part, and at suggesting edge cases you hadn't considered — treat that last one as a list to judge, not a list to accept.

Now the honest view of what it's bad at, all of which matters more. It doesn't know what our product is *supposed* to do, so it can't tell a bug from a deliberate decision. It doesn't know which failures would actually hurt a customer, so it can't prioritise. And it will cheerfully produce a test that passes without checking anything real — that's the most common failure mode by far, and you only catch it if you read what it wrote.

Three rules, and they're not negotiable.

Never submit code you can't read. If you can't explain, line by line, what your test does and what would make it fail, it doesn't go in. Ask the AI to explain its own output until you can — that's a legitimate and very fast way to learn, and it's the difference between using AI to learn faster and using it to avoid learning.

Always make a new test fail on purpose. Change the code it's checking, watch the test go red, put the code back. Thirty seconds, and it's the only proof that the test is real.

Give it real context, not a description. Paste the actual code, the actual error, the actual expected behaviour from our documentation. The quality of what comes back tracks the quality of what you put in, almost linearly. "Write a test for the login page" gets you generic nonsense; the real component, the real API response and a precise statement of the rule gets you something worth keeping.

> **The loop that works.**
>
> You decide what to check and why. AI writes the first draft. You run it, break the feature to watch it fail, fix whatever is fragile, and read every line before it goes in. You own it afterwards — if it starts failing randomly in three months, it's yours to fix.

One last thing, meant kindly. If you ever find yourself unable to explain what one of your tests does, the correct action is to delete it. A pile of tests nobody understands is a liability the team will eventually have to throw away, and nobody gets credit for the volume.

PART FIVE

What to skip, and why

Deliberately not on the roadmap. If a course, a tutorial or a job ad pushes you toward one of these, this is why we're not spending your time on it.

Certifications, at least for now
ISTQB and its relatives teach a vocabulary that's genuinely useful, wrapped in an exam that isn't. Skim the concepts from free summaries in a week — the terminology helps you read the industry — and skip the certificate. It signals nothing to us, and nothing you'd learn cramming for it beats a month of testing our actual product. Revisit it in a year or two if you ever want it on a CV.

Large hand-written test-case repositories
The classic QA job was maintaining hundreds of step-by-step cases in a spreadsheet or a tool like TestRail, and keeping them current as the product changed. That work is largely dead: repeatable checks belong in automated tests that actually run, and the rest belongs in a short checklist a person can get through in an hour. We keep a lean checklist for exactly the things a human must eyeball. Keep it lean — a document nobody reads is worse than no document.

Formal test plans and heavy process documentation
Templates asking for scope, entry criteria, exit criteria, risk matrices and sign-off sections are ceremony at our size. One page saying what we cover, what we knowingly don't, and what we're currently worried about is worth more, and gets read.

BDD frameworks — Cucumber, Gherkin, given/when/then tooling
These existed so non-programmers could write tests in structured English, with a translation layer turning that English into code. AI does that translation now, better, in both directions, with no layer to maintain. The idea underneath — describe the behaviour clearly before you test it — is permanent and free. The frameworks are overhead.

Record-and-playback and "no-code testing" tools
Selenium IDE, recorder plugins and the various no-code testing products all sell the same pitch: tests without writing code. AI already gives you that, except it gives you readable code you can fix. Recorded tests break on the first interface change and are close to undebuggable when they do. Skip the whole category.

Selenium, and the Java automation stack around it
Not bad technology — just not ours, and no longer the sensible default. Playwright does the same job with far less setup and far fewer flaky-test problems. Learn the tools we actually run, in the languages we actually use.

Deep specialisation in load and performance tooling
JMeter, k6 and friends are real skills for a real problem that isn't currently ours. Know that load testing exists and roughly what it measures. Don't take the course.

Memorising syntax, selectors and boilerplate

> Remembering the exact way to write an assertion, or select an element, or structure a page-object class, used to be a meaningful part of the skill. It's gone completely, and there's no point mourning it. Understanding *why* a selector is fragile, on the other hand, is permanent — that's judgement, not recall.

The pattern is consistent, and it's worth carrying with you when you judge anything else you come across. AI killed the **transcription** parts of QA: turning intent into documents, documents into code, code into boilerplate. It did not touch the **deciding** parts: what to test, what counts as broken, whether a passing test proves anything, and what our customers genuinely can't afford to have fail. Spend your study time on the deciding.

PART SIX

The first ninety days

What good looks like, month by month. None of it requires you to have finished the roadmap.

> **Month one — become the person who knows the product best**
>
> Use every part of our products daily, like a curious and slightly difficult customer. Read the product documentation, the calculations document and the existing testing checklist. Get accounts on staging, get the applications running with a developer's help, and run the daily check every morning. File real bug tickets from the first week — you'll find things, because fresh eyes always do, and that window closes after a couple of months.
>
> **Success: your tickets are clear enough that a developer can act on them without asking you a single question.**

> **Month two — stop reporting symptoms, start reporting causes**
>
> Live in the browser's developer tools. Learn to read Sentry. Get read-only database access and get comfortable with simple queries. Start calling the API directly with Postman. Read the Flutter tests as your textbook. Begin the permissions work from section 2.4 — one role at a time, written down.
>
> **Success: your reports name the layer that failed and the condition that triggers it.**

> **Month three — your first tests that matter**
>
> Start on the backend, where the value is highest and the tests are simplest: unit tests for the calculation rules you now understand well, then API tests for permissions. Use AI to draft, review every line, break the code to prove each test fails, and have a developer review your first few. Get at least one of them running in the pipeline.
>
> **Success: you own a small suite you can fully explain, and you have an opinion — backed by what you've seen — about what we should automate next.**

Where this is going

Our QA process isn't a finished thing you're joining; it's something we're building now, and you're expected to help shape it rather than inherit it. We're currently extending the pipeline so that automated tests actually run and actually matter on every push. The order we're leaning toward is: real unit tests on the backend calculations first, then API-level tests covering permissions and the endpoints the mobile app depends on, then a small number of Playwright journeys for the flows that must never break — logging in, running a session, opening a report.

By the end of the first ninety days you'll know more about where our product actually breaks than anyone else here. At that point we decide the rest of the process together, and your opinion will carry real weight in it.

> **Two things we'll hold you to.**
>
> Nobody expects you to know all of this in six months, and nobody here will think less of you for asking a basic question — asking early is cheaper than guessing, every time. The two standards that do apply from

> day one: never report something you haven't verified, and never ship a test you can't explain.